INDUS UNIVERSITY

Institute of Technology and Engineering

Department of Computer Science and Engineering

PROJECT REPORT

ON

PERSONAL EXPENSE TRACKER

(C++ Console-Based Application)

Submitted by

MUGDHA PANDYA

Enrollment No.: IU2541230668 (3CSE-F)

Bachelor of Technology

Computer Science and Engineering

Submitted in partial fulfilment of the academic requirements

Indus University, Ahmedabad, Gujarat, India

Academic Year 2026

CANDIDATE'S DECLARATION

I, Mugdha Pandya (IU2541230668), declare that this project report entitled "Personal Expense Tracker" describes the design and implementation of a C++ console application for recording expenses, organising spending by category, preparing monthly summaries, and managing budgets. This report is prepared for academic submission. Any source code or material supplied as a project reference should be acknowledged according to the institution's requirements.

I understand that the submitted implementation should be reviewed, tested, and explained by me before final submission.

Candidate's Signature: Mugdha

Name: Mugdha Pandya

Enrollment No.: IU2541230668

CERTIFICATE

This is to certify that the project report entitled “Personal Expense Tracker” is submitted by Mugdha Pandya (IU2541230668), student of the Bachelor of Technology programme in Computer Science and Engineering at Indus University, Ahmedabad, as an academic project report.

Project Guide: Ms. Zarna Kotak

Department of Computer Science and Engineering

Indus University, Ahmedabad

Date: 12 October, 2026 Signature: _____

Note: The guide’s name and approval details should be completed by the department or project supervisor.

ACKNOWLEDGEMENT

I would like to express my sincere gratitude to the faculty members of the Department of Computer Science and Engineering at Indus University for the academic guidance and learning environment that support practical programming work.

I am thankful to the project supervisor and classmates whose feedback can help improve the design, testing, and presentation of this application. Developing this project provides an opportunity to apply object-oriented programming, dynamic memory management, file handling, validation, and structured problem-solving in C++.

I also acknowledge the supplied C++ source file as the technical basis used to describe the system's classes, features, and implementation approach in this report.

ABSTRACT

The Personal Expense Tracker is a menu-driven C++ console application intended to help an individual record and review everyday spending. Each expense contains an identifier, monetary amount, category, date, description, and payment method. The application supports adding, displaying, searching, updating, and deleting expense records. It also produces month-based summaries, calculates category totals, and allows users to define overall or category-specific budgets.

The implementation uses object-oriented classes including Record, Expense, Category, User, Budget, FileManager, and ExpenseSystem. The abstract Record class and virtual display() function demonstrate runtime polymorphism. ExpenseSystem manages a dynamically allocated array of expenses, while standard vectors hold categories and budgets. FileManager stores profile, category, expense, and budget information in text files under a Data directory, allowing records to persist between runs.

Input validation is applied to positive amounts, category identifiers, date formatting, month formatting, and basic profile fields. Budget monitoring displays warnings when spending reaches at least 80% of a configured limit and identifies exceeded budgets. The project is an educational local application; it is not a bank-grade financial platform and does not include encryption, multi-user authentication, or a database server.

Keywords

C++, Object-Oriented Programming, Expense Management, File Handling, Dynamic Memory, Polymorphism, Budget Monitoring, Data Validation

TABLE OF CONTENTS

Section	Title
1	Introduction
1.1	Project Overview
1.2	Problem Statement and Purpose
1.3	Scope and Objectives
2	System Analysis and Design
2.1	Functional Requirements
2.2	Architecture and Data Model
2.3	Menu Structure
3	Technology and C++ Concepts
4	Implementation and Code Discussion
4.1	Class Design
4.2	Validation Logic
4.3	Persistent File Storage
4.4	Expense Operations
4.5	Monthly Reports and Budgets
5	Testing and Results
6	Limitations and Future Enhancements
7	Conclusion
8	References

1. INTRODUCTION

1.1 Project Overview

Tracking personal expenditure manually can make it difficult to understand where money is being spent, whether a monthly budget is being followed, and which categories account for the largest share of expenses. This project models a lightweight solution through a command-line interface. A user enters expense information and can later search records or generate a monthly summary.

The program is implemented as a single C++ source file organised with comments that mark logical header and implementation sections. It uses standard C++ libraries for strings, streams, formatting, vectors, algorithms, exceptions, and filesystem operations.

1.2 Problem Statement and Purpose

The core problem is the absence of a simple, structured method to record personal expenses and compare spending against planned limits. The purpose of this project is to create a maintainable console application that combines CRUD operations (Create, Read, Update, Delete), persistent local storage, category-based analysis, and budget checks while demonstrating key C++ programming concepts.

1.3 Scope

- Create expense records with automatically assigned IDs.
- Display all stored expenses in a formatted table.
- Search by expense ID, category ID, or exact date.
- Update existing expense details and delete records after confirmation.
- Calculate monthly totals and category-wise spending.
- Create overall and category-specific monthly budgets.
- Warn when 80% or more of a budget is used and identify overspending.
- Save and reload user, category, expense, and budget data from local text files.

1.4 Objectives

- Apply encapsulation through private data members and public member functions.
- Demonstrate inheritance and runtime polymorphism using Record and Expense.
- Practise constructors, destructors, const member functions, vectors, and dynamic arrays.
- Validate date, month, amount, category, description, and profile input.
- Use file streams and a delimited text format for persistent storage.
- Produce clear summaries that support basic personal budgeting.

1.5 Assumptions

- The application is run locally by one user at a time.
- Expense amounts are positive values expressed in a single, unspecified currency.
- Dates use YYYY-MM-DD and budget months use YYYY-MM.
- Category ID 0 is reserved to represent an overall budget.
- Text files are stored in the Data directory relative to the working directory.

2. SYSTEM ANALYSIS AND DESIGN

2.1 Functional Requirements

ID	Requirement	Expected behaviour
FR-01	Add expense	Validate fields, assign an ID, store the record, and save data.
FR-02	Display records	Show all expenses in aligned columns or a no-record message.
FR-03	Search	Find a record by ID, or list matches by category/date.
FR-04	Update	Replace an existing record's details after validation.
FR-05	Delete	Ask for explicit YES confirmation before removing a record.
FR-06	Monthly report	Calculate month total, category totals, and budget status.
FR-07	Budget management	Add or update a budget for a month and category.
FR-08	Profile management	Update the user name and basic email field.
FR-09	Persistence	Save and reload information from local text files.

2.2 Non-Functional Requirements

- Usability: menu options and prompts should be understandable to a beginner.
- Maintainability: responsibilities are separated into domain classes and a file manager.
- Reliability: malformed stored rows are skipped in several loading functions; validation rejects many invalid inputs.
- Portability: a C++17-compatible compiler is required because the code uses std::filesystem.
- Data safety: local files should be backed up; the current implementation does not encrypt stored information.

2.3 Architecture

Component	Responsibility
Record	Abstract base class defining the common display interface.
Expense	Represents one expense and validates its fields.
Category	Stores a category ID and category name.
User	Stores the basic profile name and email.
Budget	Stores monthly spending limits and provides remaining/exceeded checks.
FileManager	Reads and writes text files inside the Data directory.
ExpenseSystem	Coordinates user interaction, record operations, reports, and budgets.
main()	Creates ExpenseSystem, starts the menu, and handles fatal exceptions.

2.4 Data Files

File	Stored content	Format overview
users.txt	One user profile	userId|name|email
categories.txt	Expense categories	categoryId|categoryName

expenses.txt	Expense records	id|amount|categoryId|date|description|payment
budgets.txt	Budget records	budgetId|categoryId|limit|YYYY-MM

The vertical bar (|) is the field separator. FileManager::cleanField replaces bars and line breaks in text fields with spaces so that a description or name does not accidentally create additional columns or records.

2.5 Menu Structure

Option	Operation
1	Add Record
2	Display Records
3	Search Record
4	Update Record
5	Delete Record
6	Main Transaction/Process
7	Report
8	Save all data / Reload from files
9	Exit
10	Manage Budget
11	Manage User Profile

3. TECHNOLOGY AND C++ CONCEPTS

3.1 Language and Libraries

Technology	Use in this project
C++17	Core language; required for filesystem support.
iostream	Console input and output.
fstream	Reading and writing persistent text files.
string / sstream	Text storage and parsing delimited rows.
vector	Dynamic collections of categories and budgets.
algorithm	Finding maximum IDs and related utilities.
iomanip	Column alignment and two-decimal amount formatting.
filesystem	Create and address the Data directory.
stdexcept	Report invalid values through exceptions.
limits / cctype	Recover input streams and validate numeric characters.

3.2 Encapsulation

Classes keep fields private and expose controlled access through getters and setter methods. For example, Expense exposes getAmount() and getDate() rather than allowing the rest of the application to modify those fields directly. Its setDetails() function validates a complete replacement set before assigning the values.

3.3 Inheritance and Runtime Polymorphism

Record is an abstract base class because display() is pure virtual. Expense inherits from Record and overrides display(). The table-printing routine can therefore treat an Expense as a Record reference while calling the correct overridden function.

```
class Record {
public:
    Record() = default;
    virtual ~Record() = default;
    virtual void display(std::ostream& out) const = 0;
};

class Expense : public Record {
public:
    void display(std::ostream& out) const override;
};
```

3.4 Dynamic Memory Management

ExpenseSystem stores expenses in a dynamically allocated array. The initial capacity is five. When the array becomes full, growIfNeeded() allocates a larger array with twice the capacity, copies existing objects, releases the old array, and updates the pointer and capacity. The destructor releases the array with delete[]. Copy construction and copy assignment are deleted to avoid accidental shallow copies of the owning pointer.

```
void ExpenseSystem::growIfNeeded() {
    if (count < capacity) return;
    int newCapacity = capacity * 2;
    Expense* replacement = new Expense[newCapacity];
    for (int i = 0; i < count; ++i)
        replacement[i] = expenses[i];
    delete[] expenses;
```

```
        expenses = replacement;
        capacity = newCapacity;
    }
```

3.5 Exceptions and Validation

Constructors and setter methods throw std::invalid_argument when an object would contain invalid values. User-facing operations catch exceptions and display a meaningful message instead of allowing many validation errors to terminate the program.

4. IMPLEMENTATION AND CODE DISCUSSION

4.1 Expense Data Model

An Expense record stores six core fields: a positive integer ID, a positive amount, a category ID, a date, a description, and a payment method. Its display() method formats the amount to two decimal places and aligns fields for console output.

```
void Expense::setDetails(double value, int catId,
                         const std::string& expenseDate,
                         const std::string& details,
                         const std::string& payment) {
    if (value <= 0.0)
        throw std::invalid_argument("Amount must be greater than zero.");
    if (catId < 0)
        throw std::invalid_argument("Category ID cannot be negative.");
    if (!isValidDate(expenseDate))
        throw std::invalid_argument("Date must be a real date in YYYY-MM-DD format.");
    if (details.empty())
        throw std::invalid_argument("Description cannot be empty.");
    if (payment.empty())
        throw std::invalid_argument("Payment method cannot be empty.");

    amount = value;
    categoryId = catId;
    date = expenseDate;
    description = details;
    paymentMethod = payment;
}
```

The constructor also checks that the expense ID is positive before accepting the record. Calling setDetails() from the constructor centralises the field validation and reduces duplicated checks.

4.2 Date Validation

Expense::isValidDate() checks the required ten-character layout, verifies that separators appear in the correct positions, and confirms that all other characters are digits. It then checks the year and month ranges and calculates the maximum day for the selected month. Leap years are handled using the Gregorian rule.

```
bool leap = (year % 400 == 0) ||
            (year % 4 == 0 && year % 100 != 0);
if (month == 2 && leap)
    maxDay = 29;
return day >= 1 && day <= maxDay;
```

This is more robust than checking the date string's length alone because it rejects dates such as 2026-02-31. The accepted minimum year in the implementation is 1900.

4.3 Monthly Aggregation

The helper totalForMonth() iterates over stored expenses and adds an amount when the YYYY-MM prefix of its date matches the requested month. An optional category ID narrows the total to a single category; the value -1 means that all categories are included.

```
double ExpenseSystem::totalForMonth(
    const std::string& month, int categoryId) const {
```

```
    double total = 0;
    for (int i = 0; i < count; ++i) {
        if (currentMonthFromDate(expenses[i].getDate()) == month &&
            (categoryId == -1 ||
             expenses[i].getCategoryId() == categoryId)) {
            total += expenses[i].getAmount();
        }
    }
    return total;
}
```

The report calls this helper once for the overall month total and again for each category. This design is simple to understand, although it repeats scans and could be optimised for very large datasets.

4.4 File Persistence

FileManager creates the Data directory if possible. Each save operation opens its corresponding file in truncation mode and writes the current records line by line. Load operations parse each line using stringstream and reconstruct objects. Invalid rows are skipped in several loading functions, allowing other well-formed rows to load.

```
void FileManager::saveExpenses(const Expense* expenses,
                               int count) const {
    if (!storageEnabled) return;
    std::ofstream out(dataDirectory / "expenses.txt",
                      std::ios::trunc);
    if (!out) {
        storageEnabled = false;
        std::cerr << "Note: cannot write to disk; running without saving.\n";
        return;
    }
    out.precision(2);
    out.setf(std::ios::fixed);
    for (int i = 0; i < count; ++i) {
        out << expenses[i].getId() << '|'
            << expenses[i].getAmount() << '|'
            << expenses[i].getCategoryId() << '|'
            << expenses[i].getDate() << '|'
            << cleanField(expenses[i].getDescription()) << '|'
            << cleanField(expenses[i].getPaymentMethod()) << '\n';
    }
}
```

A practical limitation is that the text files are not encrypted, and the save functions generally report file failures through console messages rather than returning a success status to every caller. The application should not be used to store highly sensitive financial data.

4.5 Adding, Updating, and Deleting Records

Adding an expense

The addExpense() flow prints available categories, asks for a valid category ID and positive amount, checks the date, collects a description and payment method, creates an Expense with nextExpenseId(), expands the array if required, stores the record, and saves all data.

Updating an expense

updateExpense() first locates the selected ID. If the record exists, the user supplies replacement values. A temporary Expense object validates the replacement before it is assigned to the array position. The original ID is retained.

Deleting an expense

deleteExpense() locates the target and requires the exact confirmation text YES. It shifts subsequent array elements one position to the left and decrements count. This avoids leaving a gap in the active portion of the array.

4.6 Budget Logic

Budget records specify a month, limit, and category. Category ID 0 represents an overall monthly budget. The Budget class calculates remaining funds by subtracting actual spending from the limit and checks whether spending is greater than the limit. The interface warns at 80% usage or more.

```
double Budget::remaining(double spent) const {
    return limitAmount - spent;
}
bool Budget::isExceeded(double spent) const {
    return spent > limitAmount;
}
```

If spending exactly equals the budget, isExceeded() returns false, but the report still classifies it as a warning because the 80% threshold has been reached. The label ‘EXCEEDED’ is reserved for spending strictly above the limit.

4.7 Monthly Report

showReport() validates the selected month, displays the total expenditure, prints category totals where spending is greater than zero, and then lists applicable budgets with the remaining amount and a status. This produces a compact text-based overview of spending.

4.8 Profile Management

The User class stores a user ID, name, and email. setProfile() rejects an empty name and requires the email string to contain an @ character. This is basic validation only; it does not verify that the address exists or follows every email-format rule.

4.9 Program Entry Point

```
int main() {
    try {
        ExpenseSystem system;
        system.run();
    } catch (const std::exception& ex) {
        std::cerr << "Fatal application error: "
                  << ex.what() << '\n';
        return 1;
    }
    return 0;
}
```

The main function creates the system object and starts the menu loop. When the object goes out of scope, its destructor releases the dynamically allocated expense array.

5. TESTING AND RESULTS

The following table is a proposed test plan derived from the implementation. These are expected outcomes, not a claim that every case has been executed on a particular machine. Record actual results after compiling and running the program.

Test	Input / action	Expected result
T-01	Add a valid expense	A new ID is assigned and record is saved.
T-02	Enter amount 0 or a negative value	Input helper or object validation rejects the amount.
T-03	Enter an invalid category ID	Operation reports that the category does not exist.
T-04	Enter date 2026-02-31	Date validation rejects the invalid day.
T-05	Enter date 2024-02-29	Leap-year date is accepted.
T-06	Search by an existing ID	Matching expense is displayed.
T-07	Search for an unknown ID	No matching expense message is displayed.
T-08	Update a valid record	Replacement details are validated and saved.
T-09	Delete then answer NO	Deletion is cancelled.
T-10	Delete then type YES	Record is removed from the active list and data is saved.
T-11	Create a monthly budget	Budget is stored for the selected month/category.
T-12	Spend at least 80% of limit	Budget warning is displayed.
T-13	Spend above limit	Budget status reports exceeded.
T-14	Restart after saving	Stored profile, categories, expenses, and budgets load from files.
T-15	Enter malformed menu input	Input helper clears the stream and requests a whole number.

5.1 Compilation

The source file uses std::filesystem, so compile it with a C++17-capable compiler. A typical command is:

```
g++ -std=c++17 -Wall -Wextra -pedantic finalcodebyalay.cpp -o ExpenseTracker
```

On Windows, a compiler distribution such as MinGW-w64 with C++17 support can be used. Run the resulting executable from a directory where the program can create and write to the Data folder.

5.2 Expected Result Summary

- The main menu provides access to all listed operations.
- Expense records are shown with ID, amount, date, category ID, payment method, and description.
- Monthly reports total matching records and break down spending by category.
- Budget output distinguishes normal usage, warning-level usage, and overspending.
- Local data files allow information to remain available after the program exits, provided saving succeeds.

6. LIMITATIONS AND FUTURE ENHANCEMENTS

6.1 Current Limitations

- The interface is console-based and does not provide graphical charts.
- Storage uses delimited plain-text files rather than a transactional database.
- Data is not encrypted and there is no authentication or multi-user access control.
- Amounts use double, which is convenient for this academic project but can introduce floating-point rounding issues for currency calculations.
- Date and month validation checks format and ranges; month validation does not represent a full calendar object.
- Search is limited to ID, category ID, and exact date; there is no free-text search or date range filter.
- The application has no export-to-CSV, backup/restore workflow, recurring transactions, or income tracking.
- Some file write failures are printed as notes and may not be propagated to each operation as a formal failure.

6.2 Proposed Enhancements

1. Replace double with integer minor units (for example, paise) or a decimal-safe money representation.
2. Use SQLite with transactions, indexes, and schema constraints for stronger persistence.
3. Add password-protected profiles, encryption at rest, and careful access control if the project is extended beyond classroom use.
4. Provide date-range filters, sorting, keyword search, and pagination.
5. Add income records and calculate net cash flow and savings.
6. Export monthly reports to CSV or PDF.
7. Build a GUI or web interface while retaining the existing domain classes.
8. Add automated unit tests for date validation, budget thresholds, ID generation, and file recovery.
9. Use safer ownership such as std::vector<Expense> or std::unique_ptr<Expense[]> to reduce manual memory-management risks.
10. Improve file operations with temporary files and atomic replacement to reduce the chance of partial data loss.

7. CONCLUSION

The Personal Expense Tracker demonstrates how core C++ techniques can be combined to solve a practical record-management problem. The application includes expense CRUD operations, category-based organisation, date validation, monthly summaries, budget monitoring, and local persistence. Its class structure demonstrates encapsulation, inheritance, virtual functions, exception handling, and dynamic memory management.

The project is suitable as an educational console application and provides a foundation for more advanced financial tracking. Before real-world use, stronger persistence, money representation, error reporting, automated tests, and data security should be added. The source should be compiled and the test plan executed before claiming verified results.

8. REFERENCES

11. ISO C++ / cppreference.com — C++ language and standard library reference: https://en.cppreference.com/
12. GNU Compiler Collection documentation — GCC and C++ compilation options: https://gcc.gnu.org/onlinedocs/
13. C++ source file supplied for this project: finalcodebyalay(1).cpp.
14. Project report layout reference supplied in the conversation: Bank Account Management System Project Report.

Academic note: Verify the required citation style and institutional report template before submission.